

Daily Load Forecasting Competition: July 2011

ENV 797: Time Series Analysis for Energy and Environment Applications

Mingjie Wei, Abrao Pereira

April 24, 2026

Contents

1	GitHub Repository	3
2	Problem Statement	3
2.1	Objective	3
2.2	Data Context	3
2.3	Forecasting Task	3
3	Data and Preprocessing	3
3.1	Raw Data Description	3
3.2	Data Wrangling	4
3.3	Construction of Time Series Objects	4
3.4	Exploratory Analysis	5
4	Experimental Design	9
4.1	Training, Validation, and Final Forecasting Setup	9
4.2	Model Development Strategy	10
4.3	Evaluation Criteria	10
4.4	Reproducibility and Workflow	10
5	Modeling Results: Top 5 Models	10
5.1	Model 1: TBATS (dual seasonality: weekly + annual)	10
5.2	Model 2: Seasonal Naive (as baseline)	12
5.3	Model 3: SARIMA	13
5.4	Model 4: STL + ETS	15
5.5	Model 5: NNETAR	16
6	Comparative Summary of the Five Models	18
6.1	Performance Comparison Table	18
6.2	Cross-Model Interpretation	18
6.3	Final Model Selection	18
7	Discussion	19
7.1	Key Findings	19
7.2	Role of Weather Variables	19
7.3	Limitations	19
7.4	Future Improvements	19
8	Conclusion	20

9	References and Acknowledgments	20
9.1	References	20
9.2	AI Use Disclosure	20

1 GitHub Repository

Repository link: Wei_Pereira_ENV797_TSA_ForecastCompetition_S26

2 Problem Statement

2.1 Objective

This project aims to forecast daily residential electricity demand for July 2011 using historical behind-the-meter load data. The task is framed as a time series forecasting competition in which past demand patterns, as well as temperature and relative humidity, may improve predictive accuracy.

2.2 Data Context

Three datasets are provided for the competition:

- **Load data:** hourly electricity demand (Wh) for a single residential meter from January 2005 to June 2011.
- **Temperature data:** hourly temperature readings (°F) from nearby weather stations covering the same period.
- **Humidity data:** hourly relative humidity readings (%) from nearby weather stations covering the same period.

The load series is the target variable. Temperature and humidity are optional external predictors.

2.3 Forecasting Task

The hourly series were aggregated to daily averages before modeling. Multiple candidate models were trained, validated on July 2010 (hold-out), and compared. The best-performing model was then retrained on the full dataset (through June 2011) to generate forecasts for July 1 to 31, 2011.

3 Data and Preprocessing

3.1 Raw Data Description

```
hour_cols <- paste0("h", 1:24)

load_raw <- read_excel("data/load.xlsx") |>
  mutate(date = as.Date(date))

temp_raw <- read_excel("data/temperature.xlsx") |>
  mutate(date = as.Date(date))

hum_raw <- read_excel("data/relative_humidity.xlsx") |>
  mutate(date = as.Date(date))

cat("Load rows:", nrow(load_raw), "| Date range:",
    as.character(min(load_raw$date)), "to", as.character(max(load_raw$date)), "\n")

## Load rows: 2372 | Date range: 2005-01-01 to 2011-06-30

cat("Temp rows:", nrow(temp_raw), "\n")

## Temp rows: 56922
```

```
cat("Humidity rows:", nrow(hum_raw), "\n")
```

```
## Humidity rows: 56921
```

3.2 Data Wrangling

Each dataset is stored in wide format with one row per day and columns h1 to h24 (or per station). We compute daily averages by row-wise mean across all hourly or station columns.

```
# Load: daily mean across 24 hours
daily_load <- load_raw |>
  mutate(load_day = rowMeans(across(all_of(hour_cols)), na.rm = TRUE)) |>
  select(date, load_day) |>
  arrange(date)

# Temperature: daily mean across all weather stations
temp_station_cols <- setdiff(names(temp_raw), c("date", "hr"))
daily_temp <- temp_raw |>
  mutate(temp_mean = rowMeans(across(all_of(temp_station_cols)), na.rm = TRUE)) |>
  group_by(date) |>
  summarise(temp_day = mean(temp_mean, na.rm = TRUE), .groups = "drop")

# Humidity: daily mean across all weather stations
hum_station_cols <- setdiff(names(hum_raw), c("date", "hr"))
daily_hum <- hum_raw |>
  mutate(hum_mean = rowMeans(across(all_of(hum_station_cols)), na.rm = TRUE)) |>
  group_by(date) |>
  summarise(hum_day = mean(hum_mean, na.rm = TRUE), .groups = "drop")

# Join all three daily series
daily <- daily_load |>
  left_join(daily_temp, by = "date") |>
  left_join(daily_hum, by = "date") |>
  arrange(date)

cat("Daily rows:", nrow(daily), "| Missing load:", sum(is.na(daily$load_day)), "\n")
```

```
## Daily rows: 2372 | Missing load: 0
```

We aggregated hourly load observations to daily averages by taking the mean of the 24 hourly values for each day. Temperature and humidity were similarly averaged across all available weather stations and across the 24 hours of each day.

3.3 Construction of Time Series Objects

```
# msts() captures both weekly (7) and annual (365.25) seasonality.
# Start: January 1, 2005 (day 1).
daily_ts <- msts(
  daily$load_day,
  seasonal.periods = c(7, 365.25),
  start = c(2005, 1)
)
```

We used `msts()` rather than `ts()` because daily electricity demand exhibits two overlapping seasonal patterns: a weekly cycle (weekday vs. weekend effect) and an annual cycle (summer cooling peaks, winter heating

peaks).

3.4 Exploratory Analysis

3.4.1 Raw Time Series

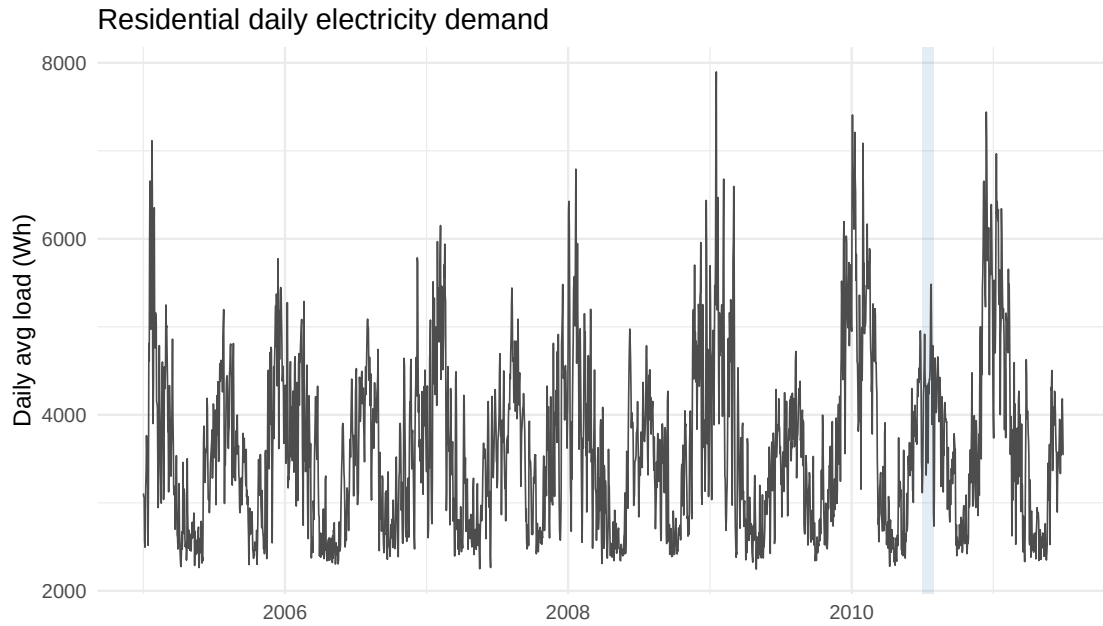


Figure 1: Daily average electricity demand from 2005 to 2011. The shaded band marks the July 2010 validation window.

3.4.2 Annual Seasonal Pattern

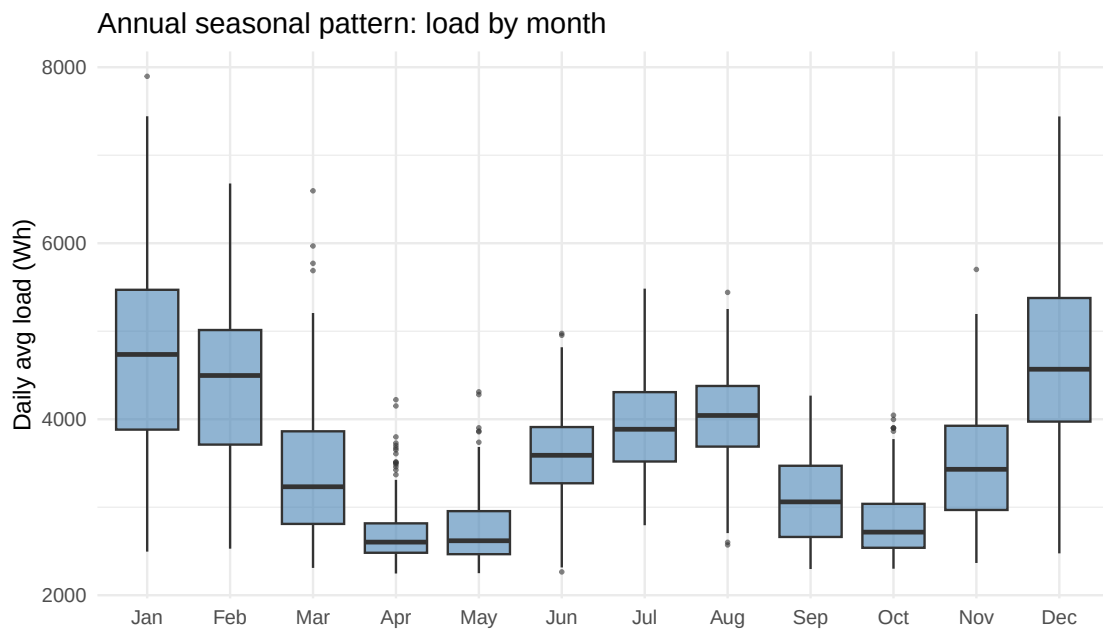


Figure 2: Distribution of daily load by calendar month, pooled across all years from 2005 to 2011. Boxes span the interquartile range; whiskers extend to 1.5 IQR.

3.4.3 Weekly Seasonal Pattern

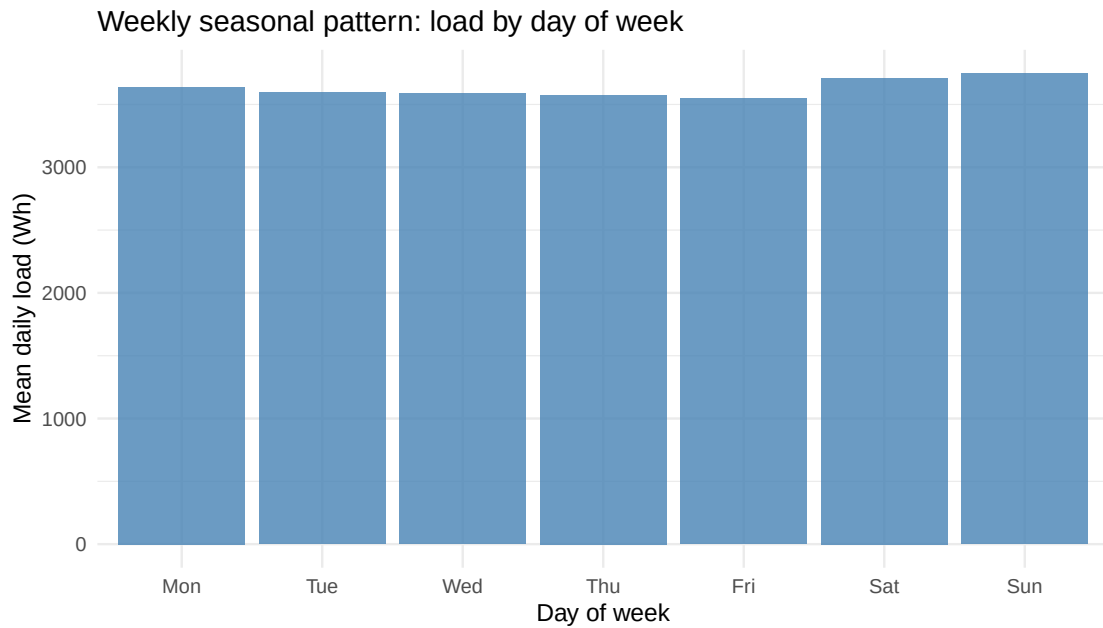


Figure 3: Average daily load by day of week (Mon to Sun), pooled across all years.

3.4.4 MSTL Decomposition

The `mstl()` function applies a Loess-based decomposition that simultaneously extracts weekly and annual seasonal components, a trend, and a remainder.

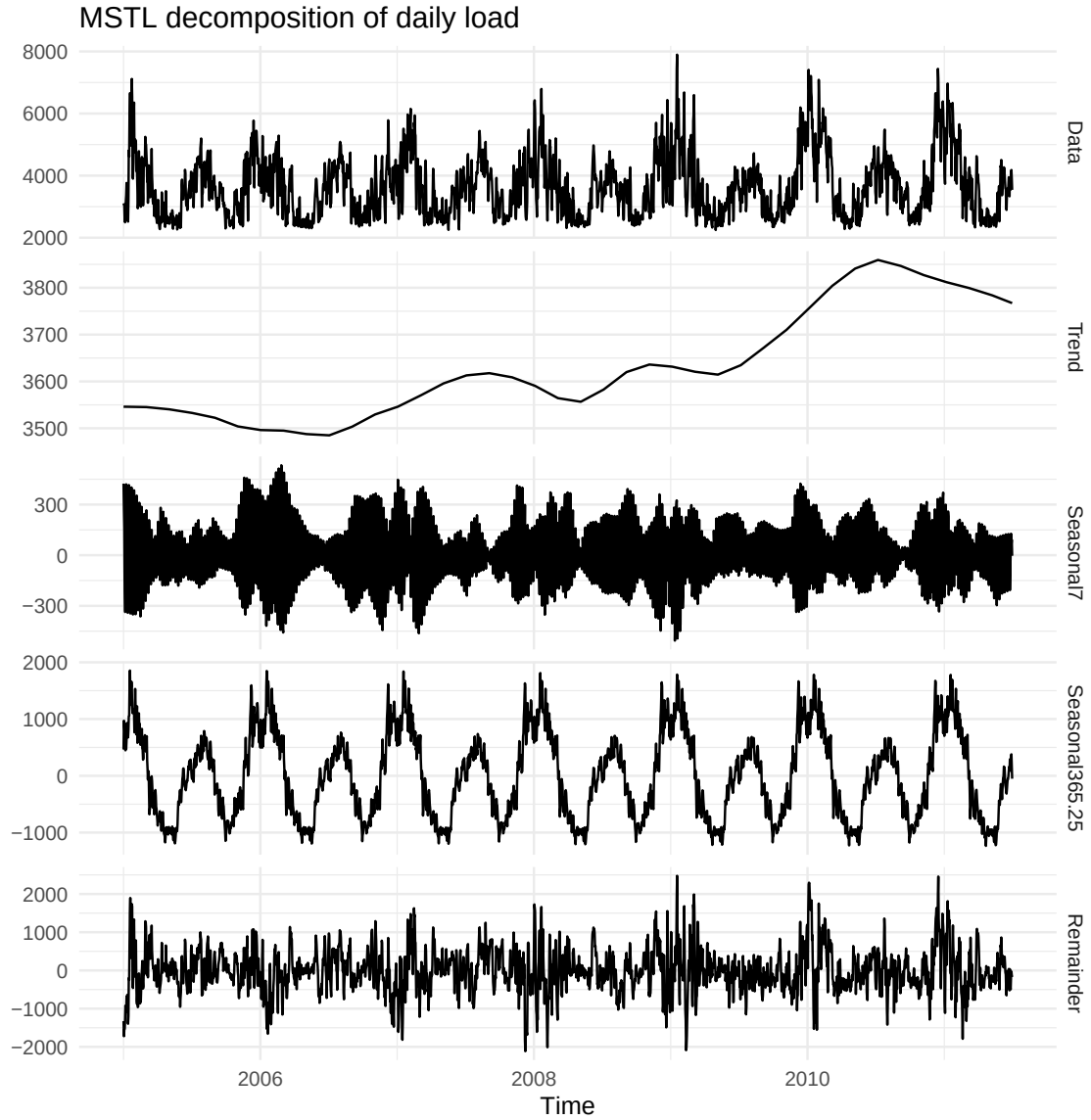


Figure 4: MSTL decomposition of the daily load series into trend, weekly seasonal, annual seasonal, and remainder components.

3.4.5 Autocorrelation Structure (ACF and PACF)

The ACF and PACF are computed on the **training series** (through 2010-06-30) to avoid look-ahead leakage. Lags extend to 56 days (~ 8 weeks) to reveal the weekly periodicity at multiples of lag 7.

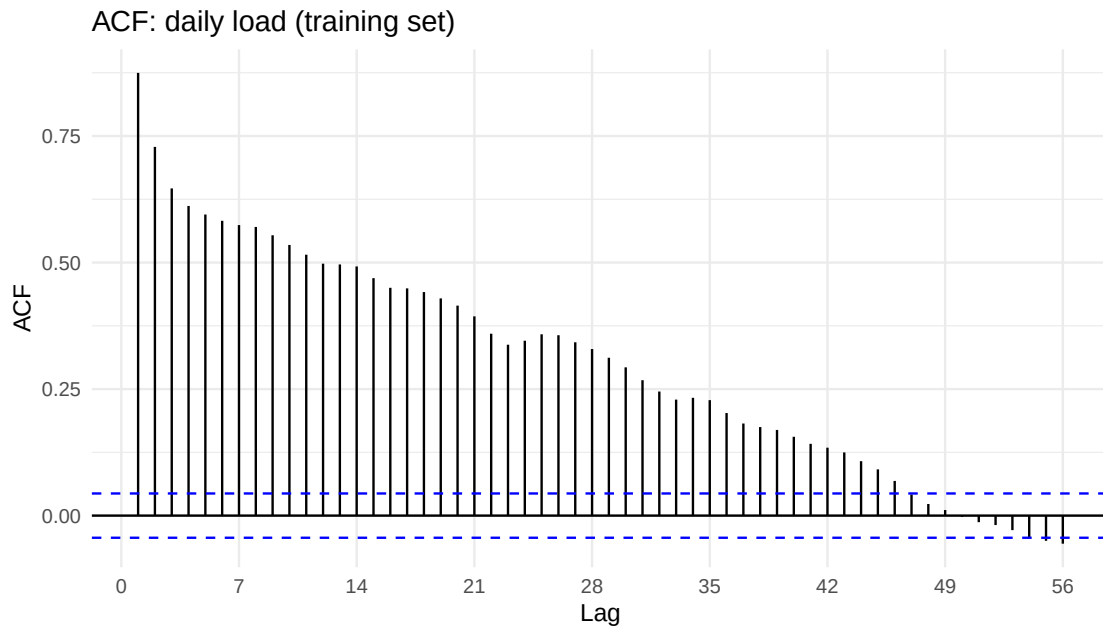


Figure 5: Sample ACF of daily load for the training period from 2005-01-01 to 2010-06-30, using lags 1 to 56. Dashed lines mark the 95% confidence bounds under the null of no autocorrelation.

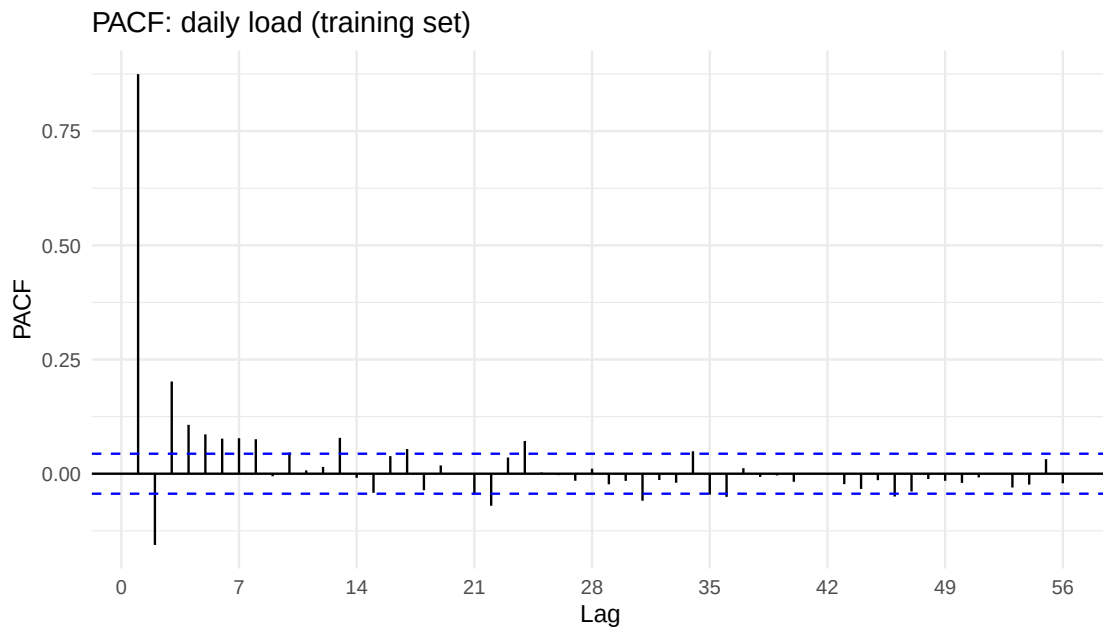


Figure 6: Sample PACF of daily load for the training period from 2005-01-01 to 2010-06-30, using lags 1 to 56.

3.4.6 Relationship with Temperature

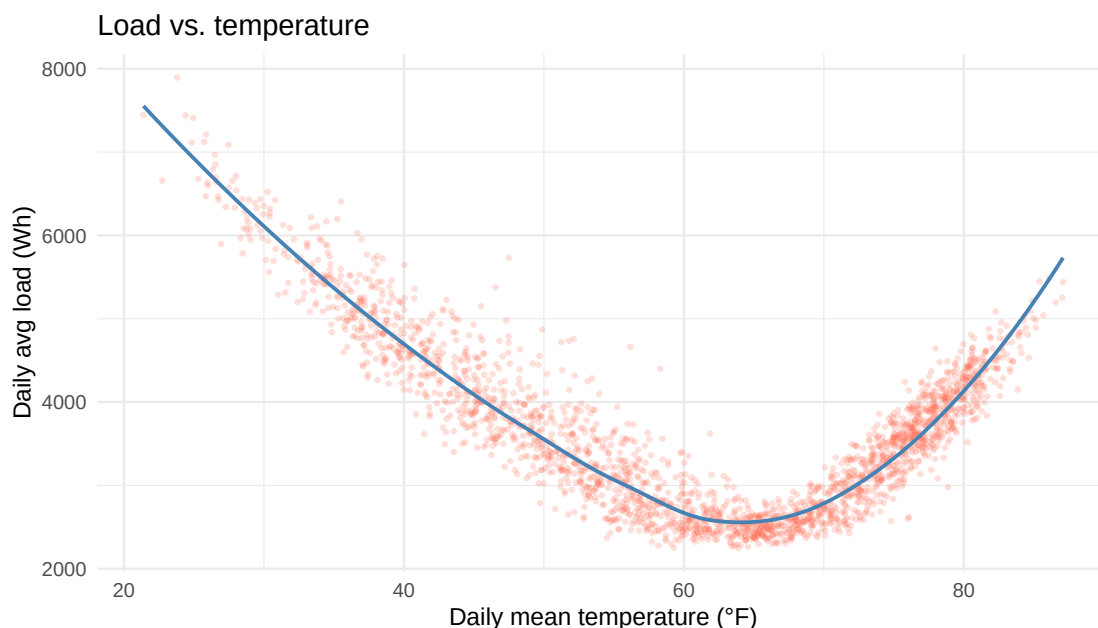


Figure 7: Scatter plot of daily load versus daily mean temperature ($^{\circ}\text{F}$). The blue curve is a LOESS smoother.

3.4.7 Summary of Key Patterns

- **Trend:** The MSTL trend component reveals a broadly stable mean load from 2005 to 2011 with no strong long-run drift, though mild inter-year fluctuations are visible. These shifts are more plausibly explained by weather variation than by a structural change in household consumption behavior.
- **Annual seasonality:** Load peaks sharply in summer (June to August), consistent with air-conditioning demand, and shows a secondary elevation in winter (December to February). The monthly boxplot confirms that July is among the highest-load months, making it a challenging forecast target with elevated and variable demand.
- **Weekly seasonality:** The day-of-week plot shows a modest but clear weekly cycle, with slightly higher average demand on weekends, especially Saturday and Sunday. For a residential meter, this pattern is plausible because occupants are more likely to spend time at home on weekends.
- **Autocorrelation:** The ACF decays slowly and displays prominent spikes at multiples of lag 7 (7, 14, 21, ...), confirming strong weekly periodicity. The PACF shows notable partial autocorrelations through approximately lag 3 to 4 before dropping toward zero, which supports the inclusion of a low-order autoregressive structure in the short-run dynamics.
- **Temperature relationship:** The scatter plot and LOESS smoother reveal a clear nonlinear relationship between daily mean temperature and daily load. Demand is lowest around mild temperatures and rises at both cooler and hotter temperatures, with a pronounced increase above approximately 60°F . This pattern motivates the use of temperature as a potential external predictor in regression-based models.

4 Experimental Design

4.1 Training, Validation, and Final Forecasting Setup

We followed the competition structure when partitioning the data:

Period	Dates	Purpose
Training	2005-01-01 to 2010-06-30	Model fitting
Validation	2010-07-01 to 2010-07-31	Hold-out evaluation (MAPE)
Final training	2005-01-01 to 2011-06-30	Refit before final forecast
Forecast horizon	2011-07-01 to 2011-07-31	Kaggle submission

```

train_end <- as.Date("2010-06-30")
val_start <- as.Date("2010-07-01")
val_end   <- as.Date("2010-07-31")

train2010 <- daily |> filter(date <= train_end)
val2010   <- daily |> filter(date >= val_start, date <= val_end)
train_full <- daily |> filter(date <= as.Date("2011-06-30"))

cat("Training rows:", nrow(train2010),
    "| Validation rows:", nrow(val2010),
    "| Full-training rows:", nrow(train_full), "\n")

```

```
## Training rows: 2007 | Validation rows: 31 | Full-training rows: 2372
```

4.2 Model Development Strategy

We evaluated models spanning a simple benchmark, linear seasonal methods, and more flexible multi-seasonal or nonlinear approaches. TBATS is presented first because it was our final selected model. Seasonal Naive is then included as the benchmark. SARIMA, STL + ETS, and NNETAR follow as alternative approaches that vary in flexibility, interpretability, and seasonal structure.

4.3 Evaluation Criteria

- **Primary metric:** Mean Absolute Percentage Error (MAPE), consistent with the Kaggle leaderboard.
- **Secondary metric:** Root Mean Squared Error (RMSE) for scale-sensitive comparison.

Performance was assessed on the July 2010 hold-out window and cross-checked against Kaggle scores after each submission.

4.4 Reproducibility and Workflow

- **GitHub** was used for code version control and collaboration.
- **Kaggle** was used to submit and compare forecast outputs.
- Individual model scripts, including `scripts/forecast_TBATS.R`, are also available in the repository.
- Only the top five models are retained in the final knitted report.

5 Modeling Results: Top 5 Models

5.1 Model 1: TBATS (dual seasonality: weekly + annual)

5.1.1 Motivation

TBATS (Trigonometric seasonality, Box-Cox transformation, ARMA errors, Trend, Seasonal components) is designed for time series with multiple seasonal periods. By using `msts()` with periods 7 and 365.25, we aim to capture both weekly and annual cycles simultaneously. This structure is well-suited to daily residential load data.

5.1.2 Specification

We fitted `tbats()` on an `msts(seasonal.periods = c(7, 365.25))` object. TBATS automatically selects the number of Fourier harmonics for each seasonal period as well as whether to apply a Box-Cox transformation.

```
y_tr_tbats <- msts(train2010$load_day, seasonal.periods = c(7, 365.25))

fit_tbats <- tbats(y_tr_tbats, use.parallel = FALSE)

fc_tbats_val <- as.numeric(forecast(fit_tbats, h = nrow(val2010))$mean)

mape_tbats <- mape_pct(val2010$load_day, fc_tbats_val)
rmse_tbats <- rmse_val(val2010$load_day, fc_tbats_val)

cat(sprintf("TBATS: Val MAPE = %.2f%% | Val RMSE = %.1f\n", mape_tbats, rmse_tbats))

## TBATS: Val MAPE = 11.34% | Val RMSE = 578.2

model_scores <- rbind(model_scores, data.frame(
  Model      = "TBATS (m = 7, 365.25)",
  Predictors = "Load only",
  Val_MAPE   = round(mape_tbats, 2),
  Val_RMSE   = round(rmse_tbats, 1)
))
```

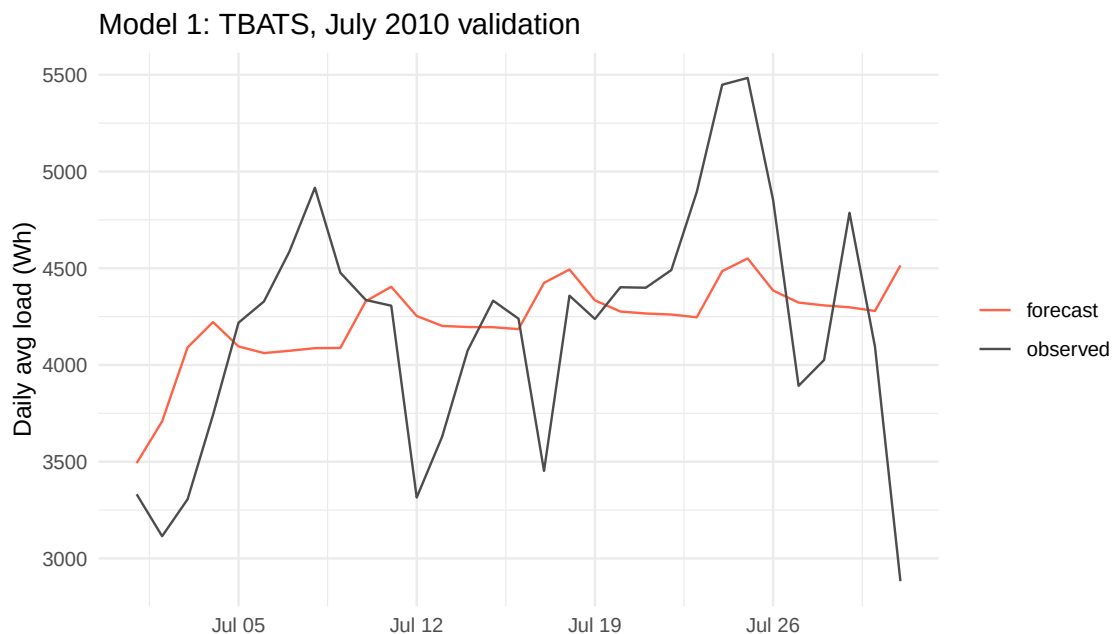


Figure 8: Model 1: TBATS forecast versus observed values for the July 2010 validation period.

5.1.3 Validation Performance

TBATS achieved a validation MAPE of **11.34%** and RMSE of **578.2** on the July 2010 hold-out.

5.1.4 Strengths and Limitations

TBATS captures dual seasonality without requiring manual specification and automatically handles the Box-Cox transformation when appropriate. Its main limitations are computational cost and a less interpretable parameter structure than simpler benchmark models.

5.2 Model 2: Seasonal Naive (as baseline)

5.2.1 Motivation

Seasonal Naive serves as our baseline because it is simple, transparent, and requires no tuning. For a daily series with strong seasonality, it provides a useful reference by projecting the previous year's same-day pattern forward. If a more sophisticated model cannot beat this benchmark, it is not adding practical forecasting value.

5.2.2 Specification

We applied `snaive()` from the `forecast` package to an `msts(seasonal.periods = c(7, 365.25))` training object. The model requires no parameter estimation.

```
y_tr_snaive <- msts(train2010$load_day,
                    seasonal.periods = c(7, 365.25))
fit_snaive  <- snaive(y_tr_snaive, h = nrow(val2010))
fc_snaive_val <- as.numeric(fit_snaive$mean)

mape_snaive <- mape_pct(val2010$load_day, fc_snaive_val)
rmse_snaive <- rmse_val(val2010$load_day, fc_snaive_val)

cat(sprintf("Seasonal Naive: Val MAPE = %.2f%% | Val RMSE = %.1f\n",
            mape_snaive, rmse_snaive))
```

```
## Seasonal Naive: Val MAPE = 19.36% | Val RMSE = 950.4
```

```
model_scores <- rbind(model_scores, data.frame(
  Model      = "Seasonal Naive",
  Predictors = "Load only",
  Val_MAPE   = round(mape_snaive, 2),
  Val_RMSE   = round(rmse_snaive, 1)
))
```

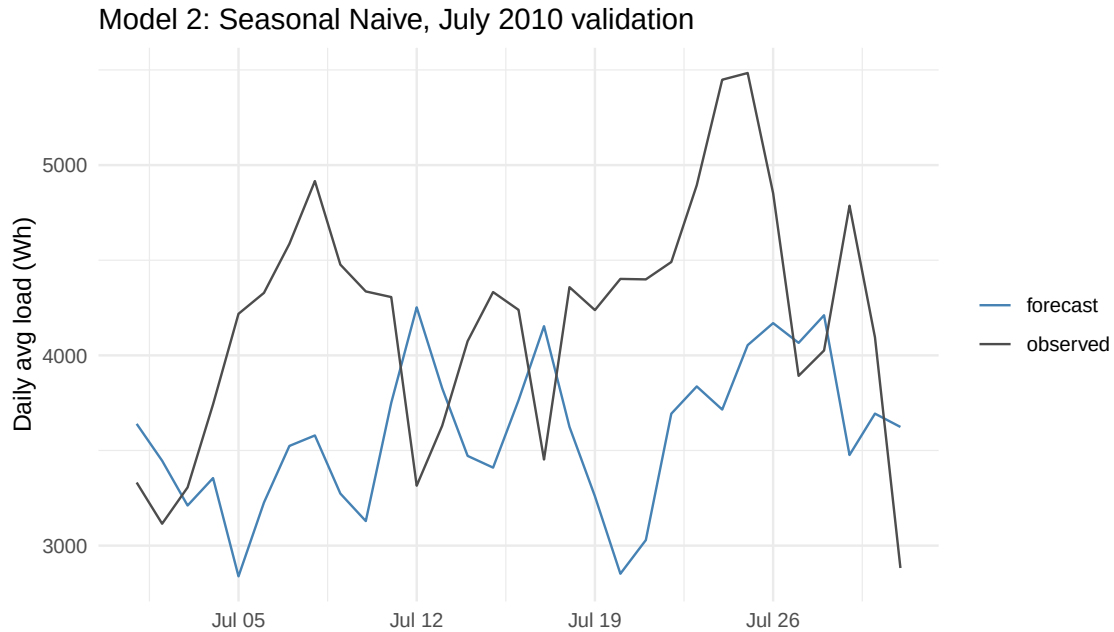


Figure 9: Model 2: Seasonal Naive forecast versus observed values for the July 2010 validation period.

5.2.3 Validation Performance

Seasonal Naive produced a validation MAPE of 19.36% and an RMSE of 950.4. This was the weakest result among the five models, confirming that the competition task requires more than a simple carry-forward seasonal benchmark.

5.2.4 Strengths and Limitations

Its main strength is transparency: the model is easy to explain and requires no parameter estimation. Its main limitation is rigidity. Because forecasts are tied closely to last year's seasonal pattern, the model cannot adapt to weather variation or gradual year-to-year shifts in load level.

5.3 Model 3: SARIMA

5.3.1 Motivation

SARIMA was included to model the strong weekly autocorrelation identified in the ACF and PACF plots. Because `auto.arima()` cannot directly fit an `msts()` object with multiple seasonal periods, we constructed a weekly `ts()` object with `frequency = 7` and focused on the dominant short-run cycle. Orders were selected automatically by AIC.

5.3.2 Specification

We used `auto.arima()` with `stepwise = TRUE` and `approximation = TRUE` on a weekly `ts` object (`frequency = 7`). The function automatically selected `ARIMA(4,0,1)(2,0,0)[7]`.

```
y_tr_sarima <- ts(train2010$load_day, frequency = 7,
                  start = c(2005, 1))

fit_sarima  <- auto.arima(y_tr_sarima,
                          stepwise    = TRUE,
```

```

                                approximation = TRUE)
lb_p_sarima  <- Box.test(residuals(fit_sarima), lag = 14, type = "Ljung-Box")$p.value

fc_sarima_val <- as.numeric(
  forecast(fit_sarima, h = nrow(val2010))$mean)

mape_sarima <- mape_pct(val2010$load_day, fc_sarima_val)
rmse_sarima <- rmse_val(val2010$load_day, fc_sarima_val)

cat(sprintf("SARIMA: Val MAPE = %.2f%% | Val RMSE = %.1f\n",
  mape_sarima, rmse_sarima))

## SARIMA: Val MAPE = 13.52% | Val RMSE = 690.4

model_scores <- rbind(model_scores, data.frame(
  Model      = "SARIMA(4,0,1)(2,0,0)[7]",
  Predictors = "Load only",
  Val_MAPE   = round(mape_sarima, 2),
  Val_RMSE   = round(rmse_sarima, 1)
))

```

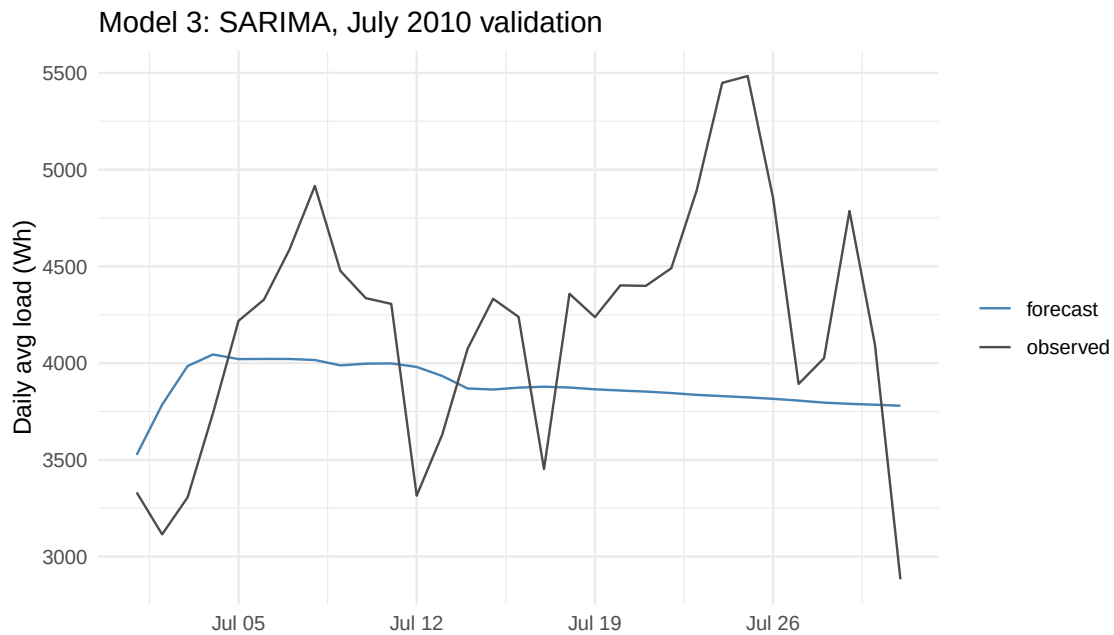


Figure 10: Model 3: SARIMA forecast versus observed values for the July 2010 validation period.

5.3.3 Validation Performance

SARIMA achieved a validation MAPE of 13.52% and an RMSE of 690.4, which was a clear improvement over the Seasonal Naive baseline. A Ljung-Box test on the residuals at lag 14 gave a p-value of **0.554**, indicating no strong evidence of remaining autocorrelation at that horizon.

5.3.4 Strengths and Limitations

SARIMA performs well because it captures short-run weekly dependence in an interpretable parametric framework. Its main limitation is that it is restricted to a single seasonal period (`frequency = 7`) and therefore does not explicitly model the annual cycle, which is especially important for a July target.

5.4 Model 4: STL + ETS

5.4.1 Motivation

We included STL + ETS as a decomposition-based alternative for handling multiple seasonal patterns. The idea is to separate weekly and annual seasonality first, then fit an ETS model to the seasonally adjusted series. This approach offers a useful contrast with TBATS because it handles dual seasonality through decomposition rather than through a unified state-space structure.

5.4.2 Specification

We applied `stlf()` directly to the `msts` training object with `seasonal.periods = c(7, 365.25)`. The function performs STL decomposition and ETS fitting in a single call, automatically selecting the ETS form.

```
y_tr_stlf <- msts(train2010$load_day,
                  seasonal.periods = c(7, 365.25))
fit_stlf <- stlf(y_tr_stlf, h = nrow(val2010))
fc_stlf_val <- as.numeric(fit_stlf$mean)

mape_stlf <- mape_pct(val2010$load_day, fc_stlf_val)
rmse_stlf <- rmse_val(val2010$load_day, fc_stlf_val)

cat(sprintf("STL + ETS: Val MAPE = %.2f%% | Val RMSE = %.1f\n",
            mape_stlf, rmse_stlf))
```

```
## STL + ETS: Val MAPE = 14.49% | Val RMSE = 705.2
```

```
model_scores <- rbind(model_scores, data.frame(
  Model      = "STL + ETS",
  Predictors = "Load only",
  Val_MAPE   = round(mape_stlf, 2),
  Val_RMSE   = round(rmse_stlf, 1)
))
```

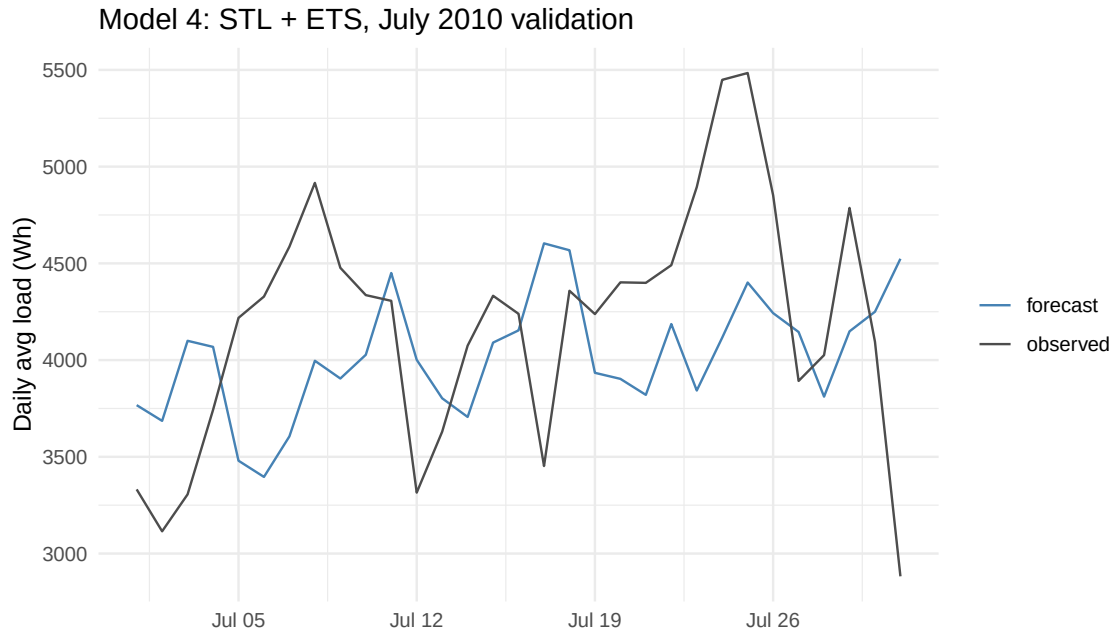


Figure 11: Model 4: STL + ETS forecast versus observed values for the July 2010 validation period.

5.4.3 Validation Performance

STL + ETS achieved a validation MAPE of 14.49% and RMSE of 705.2 on the July 2010 hold-out. While this model handles both weekly and annual seasonality, it did not perform as well as TBATS and SARIMA on this dataset.

5.4.4 Strengths and Limitations

STL + ETS handles multiple seasonal patterns in a straightforward way and offers a clear decomposition-based workflow. However, in this dataset it was less accurate than both TBATS and SARIMA, suggesting that decomposition alone was not enough to match the best-performing models.

5.5 Model 5: NNETAR

5.5.1 Motivation

We included NNETAR to test whether a nonlinear forecasting model could improve on the linear alternatives. By feeding lagged values into a single-hidden-layer neural network, NNETAR can capture nonlinear relationships that standard seasonal models may miss.

5.5.2 Specification

We applied `nnetar()` to the `msts` training object with `set.seed(123)` for reproducibility. The function automatically determined the number of input lags and hidden nodes.

```
set.seed(123)
y_tr_nnetar <- msts(train2010$load_day,
                    seasonal.periods = c(7, 365.25))
fit_nnetar <- nnetar(y_tr_nnetar)
fc_nnetar_val <- as.numeric(
```



```

forecast(fit_nnetar, h = nrow(val2010))$mean)

mape_nnetar <- mape_pct(val2010$load_day, fc_nnetar_val)
rmse_nnetar <- rmse_val(val2010$load_day, fc_nnetar_val)

cat(sprintf("NNETAR: Val MAPE = %.2f%% | Val RMSE = %.1f\n",
            mape_nnetar, rmse_nnetar))

## NNETAR: Val MAPE = 14.95% | Val RMSE = 807.4

model_scores <- rbind(model_scores, data.frame(
  Model      = "NNETAR",
  Predictors = "Load only",
  Val_MAPE   = round(mape_nnetar, 2),
  Val_RMSE   = round(rmse_nnetar, 1)
))

```

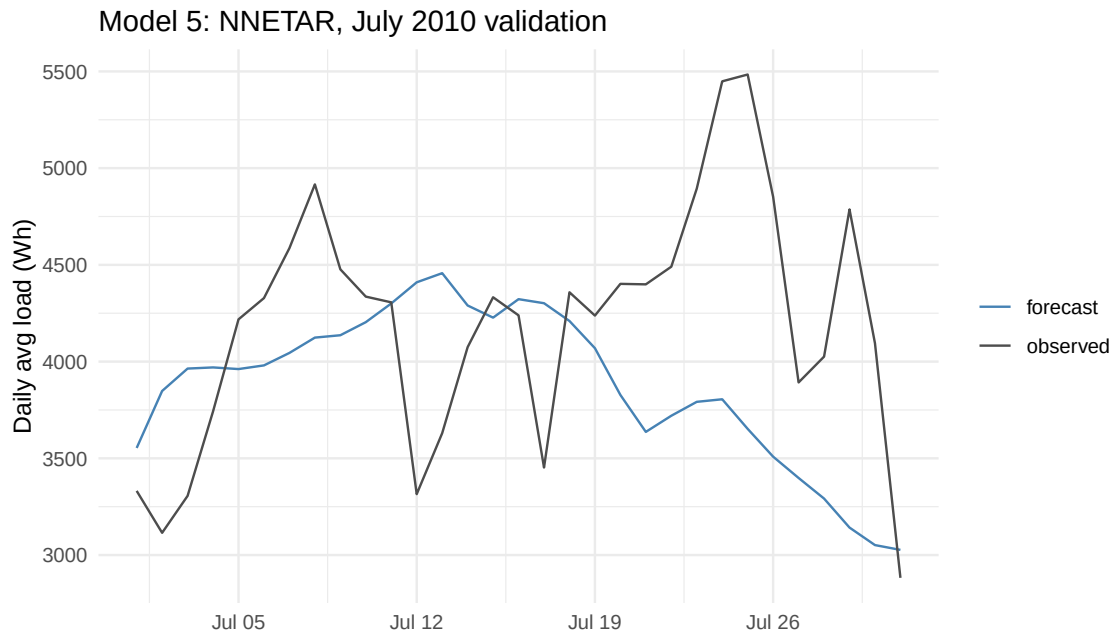


Figure 12: Model 5: NNETAR forecast versus observed values for the July 2010 validation period.

5.5.3 Validation Performance

NNETAR achieved a validation MAPE of 14.95% and an RMSE of 807.4 on the July 2010 hold-out. This was weaker than TBATS, SARIMA, and STL + ETS, suggesting that the neural network did not generalize as well as the stronger linear alternatives.

5.5.4 Strengths and Limitations

NNETAR is attractive because it does not rely on stationarity assumptions and can capture nonlinear patterns. In this application, however, that flexibility appears to have come at the cost of weaker generalization, which is consistent with mild overfitting.

6 Comparative Summary of the Five Models

6.1 Performance Comparison Table

Table 1: Validation performance of the top five models (July 2010 hold-out).

Model	Predictors	Val MAPE (%)	Val RMSE
TBATS (m = 7, 365.25)	Load only	11.34	578.2
Seasonal Naive	Load only	19.36	950.4
SARIMA(4,0,1)(2,0,0)[7]	Load only	13.52	690.4
STL + ETS	Load only	14.49	705.2
NNETAR	Load only	14.95	807.4

Table 1 shows that **TBATS achieved the best validation MAPE (11.34%)** among all five models, followed by SARIMA (13.52%), STL + ETS (14.49%), NNETAR (14.95%), and Seasonal Naive (19.36%). Among the two models submitted to Kaggle, TBATS also produced the lower public score, at 9.98% versus 14.37% for SARIMA. The gap between TBATS and the other models underscores the value of explicitly capturing both weekly and annual seasonality.

6.2 Cross-Model Interpretation

- **Seasonality capture:** TBATS and STL + ETS both explicitly model dual seasonality (periods 7 and 365.25), while SARIMA is restricted to a single weekly period (frequency = 7) and Seasonal Naive uses a simpler seasonal carry-forward rule. NNETAR implicitly learns seasonal patterns from lagged inputs. TBATS achieved the best results, which supports the importance of modeling both weekly and annual structure for a July target. At the same time, STL + ETS did not surpass SARIMA, so dual seasonality alone was not sufficient; the specific modeling framework also mattered.
- **Responsiveness to short-term changes:** TBATS and SARIMA showed better week-to-week tracking in the July 2010 validation period, as their ARMA components adapt to recent autocorrelation structure. Seasonal Naive was the most rigid benchmark, while STL + ETS adapted more slowly than the two strongest models.
- **Usefulness of weather covariates:** None of the five models incorporated temperature or humidity as explicit predictors. Given the clear nonlinear load-temperature relationship identified in the EDA, dynamic harmonic regression with weather regressors represents a clear avenue for further improvement.
- **Stability vs. flexibility:** With only 31 validation days, models with fewer parameters (TBATS, SARIMA) were less likely to overfit than the neural network. NNETAR had the second-highest RMSE (807.4), consistent with overfitting the training series and failing to generalize to the July validation window.

6.3 Final Model Selection

```
# Retrain TBATS on the full dataset (through 2011-06-30)
# and generate July 2011 forecasts.
# Submission CSVs are produced by the dedicated scripts in scripts/.

y_full_tbats <- msts(train_full$load_day, seasonal.periods = c(7, 365.25))

fit_final <- tbats(y_full_tbats, use.parallel = FALSE)

fc_july2011 <- as.numeric(forecast(fit_final, h = 31)$mean)
```

```

tpl <- read.csv("data/submission_template.csv", stringsAsFactors = FALSE)
tpl$load <- round(fc_july2011, digits = 2)

head(tpl)

```

```

##      date    load
## 1 2011-07-01 3598.77
## 2 2011-07-02 3886.33
## 3 2011-07-03 4038.80
## 4 2011-07-04 3944.31
## 5 2011-07-05 3896.60
## 6 2011-07-06 3943.40

```

We selected **TBATS** for the final submission because it achieved the lowest validation MAPE (11.34%) and also outperformed SARIMA on Kaggle, with a public score of 9.98% versus 14.37%. Its ability to model weekly and annual seasonality jointly, without extensive manual tuning, made it well-suited to the dual-seasonal structure of daily residential load data.

7 Discussion

7.1 Key Findings

Daily residential electricity demand exhibits two dominant seasonal cycles: a weekly pattern driven by day-of-week behavior and a strong annual pattern with a summer peak (June to August) consistent with air-conditioning load. TBATS delivered the best overall accuracy, with a local validation MAPE of 11.34% and a Kaggle score of 9.98%, demonstrating that flexible dual-seasonal modeling generalizes well beyond the training period. SARIMA also performed well, indicating that weekly structure matters substantially, but the gap between SARIMA and TBATS suggests that annual seasonality still adds important information for a July forecast. The nonlinear NNETAR model did not outperform the stronger linear alternatives, suggesting that the daily series is already well described by seasonal and trend structure at this level of aggregation.

7.2 Role of Weather Variables

None of the five models in this competition incorporated temperature or humidity as explicit predictors. However, the EDA (Section 3.4) revealed a clear nonlinear relationship between daily mean temperature and daily load, with demand rising at both cooler and hotter temperatures and especially above approximately 60 °F. This pattern suggests that weather-augmented models, such as dynamic harmonic regression with temperature as a regressor, could materially reduce MAPE, particularly during extreme heat events that shift load above historical seasonal norms. Adding such covariates is a clear priority for future modeling work.

7.3 Limitations

- **Short validation window:** A single 31-day July hold-out may not fully characterize model performance across seasons.
- **Single-meter data:** Results may not generalize to aggregate or grid-level load.
- **External predictors:** Only daily averages were prepared for the weather covariates; higher-frequency summaries such as hourly or maximum temperature may prove more informative.

7.4 Future Improvements

- **Weather-augmented models:** Incorporating daily temperature (and possibly maximum temperature) as an external regressor in a dynamic harmonic regression or ARIMAX framework to capture load spikes driven by heat events.

- **Fourier-term regression:** Using Fourier terms for the annual cycle within an ARIMA-errors regression (`auto.arima` with `xreg`) to combine interpretable parametric structure with dual-seasonal flexibility.
- **Ensemble forecasting:** Combining point forecasts from TBATS and SARIMA (e.g., weighted average or stacking) to reduce individual model variance.
- **Gradient boosting with calendar features:** Exploring tree-based models (XGBoost, LightGBM) with engineered features such as day of week, month, holiday indicators, lagged load, and rolling temperature averages as a fully non-parametric alternative.

8 Conclusion

This project developed and compared five time series forecasting models, namely TBATS, Seasonal Naive, SARIMA, STL + ETS, and NNETAR, to predict daily residential electricity demand for July 2011. Exploratory analysis confirmed two dominant seasonal cycles (weekly and annual) and a clear nonlinear relationship between load and temperature. Across all five models, TBATS achieved the best local validation performance, with a July 2010 hold-out MAPE of 11.34%. Among the two submitted models, TBATS also outperformed SARIMA on Kaggle, scoring 9.98% versus 14.37%. These results demonstrate that explicitly modeling dual seasonality is critical for daily residential load forecasting, and that automated multi-seasonal methods such as TBATS provide a strong and practical baseline. Future work should investigate weather-augmented regression models and ensemble approaches to further reduce forecast error.

9 References and Acknowledgments

9.1 References

- Hyndman, R. J., & Athanasopoulos, G. (2021). *Forecasting: Principles and Practice* (3rd ed.). OTexts. <https://otexts.com/fpp3/>
- R Core Team (2024). *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing.
- Hyndman, R., et al. (2024). *forecast: Forecasting Functions for Time Series and Linear Models*. R package version 8.x. <https://pkg.robjhyndman.com/forecast/>
- De Livera, A. M., Hyndman, R. J., & Snyder, R. D. (2011). Forecasting time series with complex seasonal patterns using exponential smoothing. *Journal of the American Statistical Association*, 106(496), 1513-1527.
- Hyndman, R. J., & Khandakar, Y. (2008). Automatic time series forecasting: The forecast package for R. *Journal of Statistical Software*, 27(3), 1-22.

9.2 AI Use Disclosure

LLMs were used to assist with code structuring, report organization, and language polishing. All modeling decisions, parameter choices, interpretation of results, and final submissions were completed and verified by us.